ARTIFICIAL INTELLIGENCE · BS COMPUTER SCIENCE · LECTURE 3

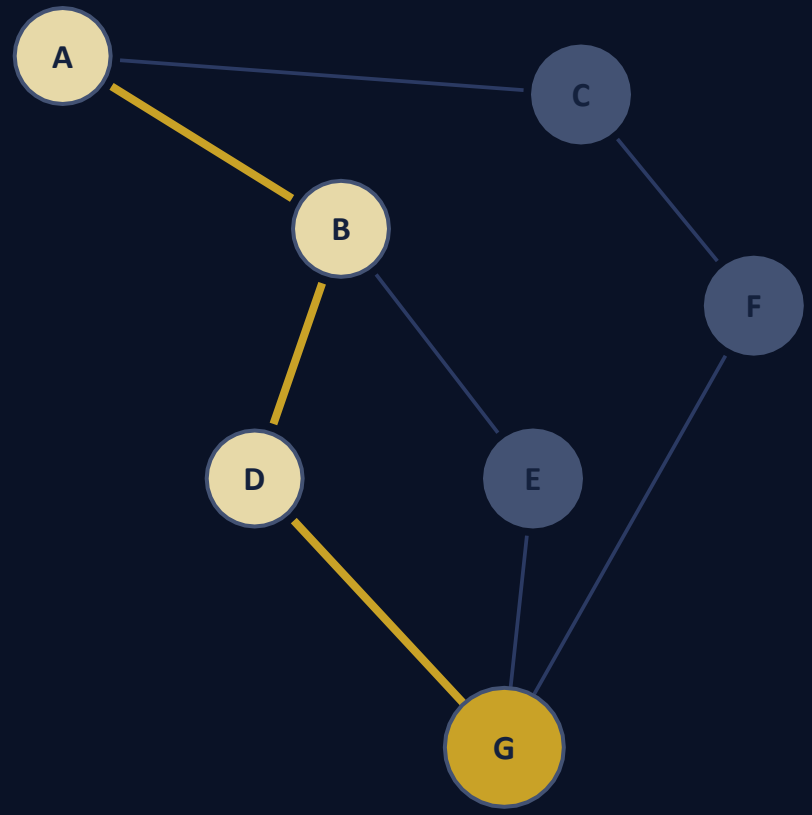
# Problem Formulation and Uninformed Search

*From Real-World Problems to Search Algorithms*

A formal foundation for how intelligent agents search for solutions — from defining a problem precisely to the first family of systematic search strategies.


Instructor **Dr. Ghulam Ali**

Course Artificial Intelligence — BS Computer Science


A → B → D → G

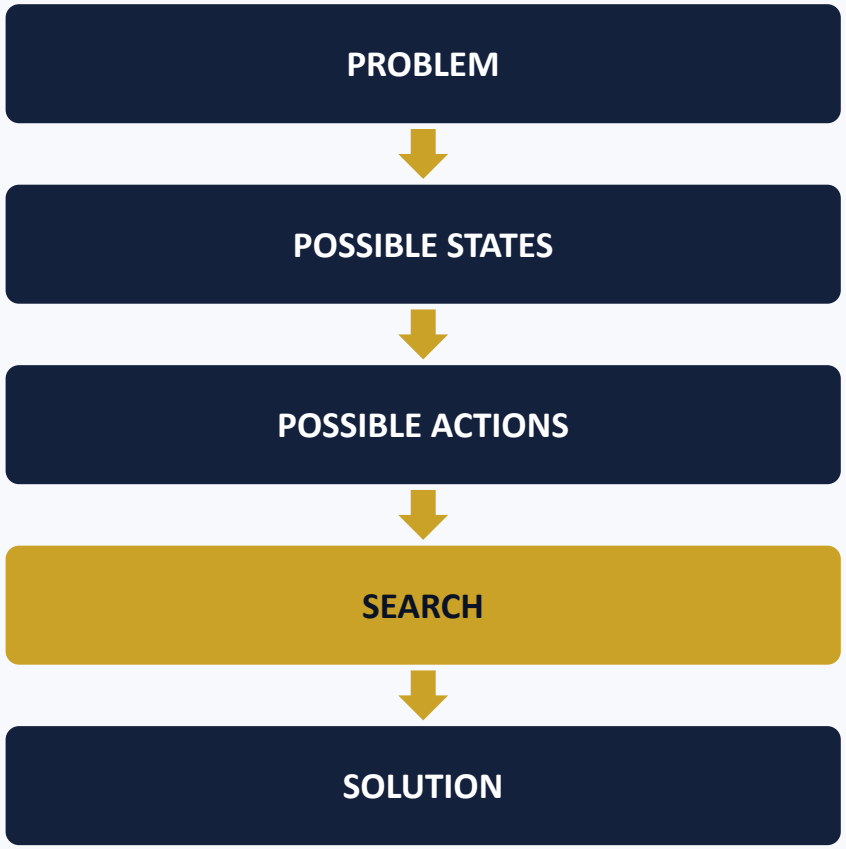
*Search explores a state space to find a path from the initial state to the goal.*

# Why Search Is Important in AI

An intelligent agent rarely reaches its goal through a single action. Usually it must choose a sequence of actions, and the number of possible sequences grows very quickly as a problem gets larger. AI search formalizes this challenge: before the agent ever acts in the real world, it explores — mentally, inside its own representation of the problem — which states are reachable and which sequence of actions leads to a goal state.

### SEARCH-BASED PROBLEM SOLVING APPEARS IN:

- Robot navigation — choosing a sequence of movements through a space that contains obstacles
- Route planning — sequencing road or flight segments from an origin to a destination
- Puzzle solving — sequencing tile or piece moves toward a target configuration (e.g. the 8-puzzle)
- Automated planning — sequencing operations so that a goal condition becomes true
- Game playing — sequencing moves in a state space that an adversary also influences
- Scheduling — sequencing task assignments while respecting time and resource constraints




*Search is the mechanism that connects a problem statement to a verified solution — it is the reasoning step performed before acting.*

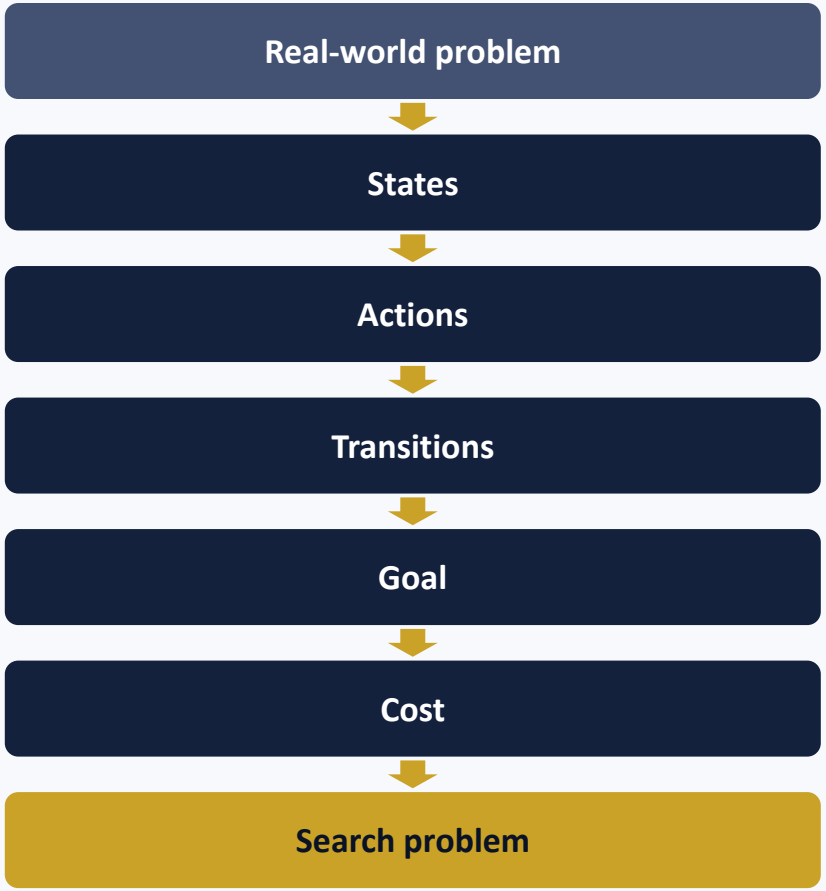
# Learning Objectives

*By the end of this lecture, students should be able to:*

1. Formulate a real-world AI problem as a formal search problem
2. Identify a problem's initial state and goal test precisely
3. Define the actions available in a state and its transition model
4. Represent a problem's state space and reason about its structure
5. Distinguish a state, a search node, a search tree, and a state-space graph
6. Explain what makes a search strategy uninformed rather than informed
7. Trace the execution of BFS, DFS, and UCS on a problem by hand
8. Analyze whether a strategy is complete, and under which assumptions
9. Analyze whether a strategy is optimal, and under which assumptions
10. Compare the time and memory requirements of BFS, DFS, and UCS
11. Select an appropriate uninformed strategy given a problem's constraints

# From Real-World Problem to AI Search Problem

A computer cannot search over a real-world situation directly — a city, a warehouse floor, or a puzzle box is not something an algorithm can manipulate. Before any searching happens, the AI system must abstract the situation into a formal representation: a precise description of the states the world can be in, the actions available, and what counts as success. This step is called problem formulation, and the quality of the abstraction directly determines whether search will succeed, and how efficiently.




e.g.

### Running example for this section

"Find a route from City A to City G." We will formulate this single sentence as a complete formal search problem across the next several slides — states, actions, transitions, goal test, and cost.

*Abstraction keeps only what is relevant to solving the problem — irrelevant details (road color, weather, driver's mood) are deliberately discarded so the state space stays searchable.*

# Formal Definition of a Search Problem

$$P = (S, A, Result, s_0, Goal, Cost)$$

*A search problem is formally defined as a 6-tuple. Every algorithm in this lecture is written purely in terms of these six components — nothing else is assumed.*

| Symbol | Name | Meaning |
|---|---|---|
| S | State space | The set of all states the problem can possibly be in |
| A(s) | Actions | The actions legally available to the agent while in state s |
| Result(s,a) | Transition model | The state produced when action a is applied in state s |
| $s_0$ | Initial state | The state the agent starts in, before any action is taken |
| Goal(s) | Goal test | A test returning true if state s satisfies the objective |
| Cost(s,a,s') | Path / step cost | The numeric cost of moving from s to s' via action a |

*We will devote one full slide to each component, starting with the initial state.*

# The Initial State ($s_0$)

The initial state describes precisely where the agent begins — its situation before any action has been taken. Because every path the search explores is measured relative to this starting point, $s_0$ must be defined with the same precision as every other state in the state space, not left as an informal description.

1 **Robot navigation**

The robot is in Room A, facing north

2 **Route planning**

The traveler is currently in Islamabad

3 **8-puzzle**

Tiles are arranged in a specific starting configuration

4 **Chess**

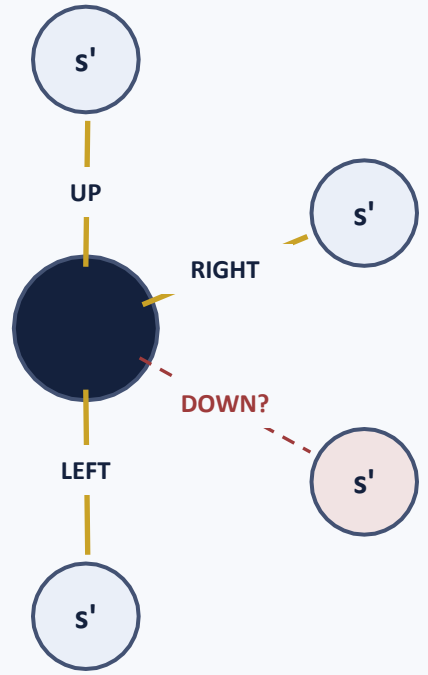
The board is at its current, specific position

*A precisely defined initial state is what allows path cost g(n) to be measured consistently for every node in the search.*

# Actions — A(s)

A(s) defines the set of actions that are legally available while the agent is in a specific state s. Actions are not global — the set of legal actions can change from state to state (a robot cannot move through a wall it is currently facing, even though 'move forward' is a valid action elsewhere in the state space).

- **Robot navigation:** UP, DOWN, LEFT, RIGHT — a subset legal at any given cell
- **Route planning:** Travel(CityX → CityY) for each road connected to the current city
- **8-puzzle:** Move the blank tile UP, DOWN, LEFT, or RIGHT into an adjacent cell



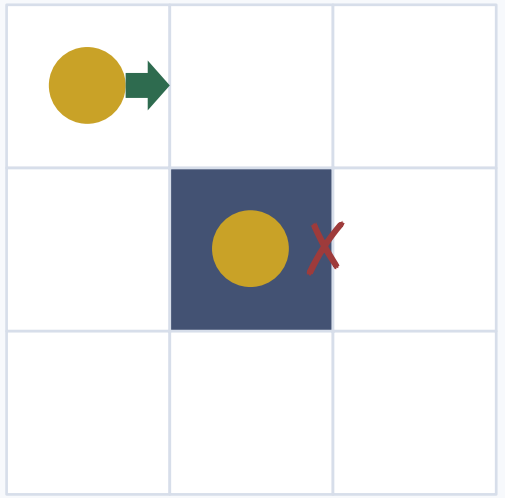

*A(s) = { UP, RIGHT, LEFT } — DOWN is not legal from this particular state s*

# The Transition Model — Result(s, a)

The transition model Result(s, a) determines exactly which state results from applying action a while in state s. It is what makes an action's effect precise and predictable — without it, 'move right' would be an instruction with no defined outcome.

```
Current state:    s  = (2, 3)
Action taken:     a  = RIGHT
Result(s, a)   =  (2, 4)
```

*Not every action is necessarily legal in every state — a well-formed problem restricts Result(s,a) to states in A(s), and never silently produces an undefined or invalid state.*



**Valid:** Result(s,RIGHT) moves into an open cell

**Invalid:** RIGHT is blocked by a wall — not in A(s)

# The Goal Test — Goal(s)

Goal(s) is a test — a function returning true or false — that determines whether a given state satisfies the problem's objective. Writing it as a test, rather than as a single fixed state, allows a problem to have many acceptable goal states.

| Problem | Goal test Goal(s) |
|---|---|
| Robot navigation | current position = destination coordinates |
| 8-puzzle | current tile configuration = target configuration |
| Route planning | current city = destination city |
| Chess (checkmate) | current position is any configuration where the opponent is checkmated |

≠

**Goal state vs. Goal test — an important distinction**

A goal state is one particular state that satisfies the objective. Goal(s) is the procedure that checks whether ANY given state qualifies. A problem may have exactly one goal state (a specific city) or many (any checkmating position) — the goal test handles both uniformly.

# Path Cost — g(n)

Path cost, written g(n), is the cumulative cost of the sequence of actions from the initial state to node n. It lets us compare two different solutions that both reach a goal, and decide which one is preferable — not merely whether a solution exists.

1

### Uniform / unit-cost problems

Every action costs exactly 1 (e.g., each grid move). Here, path cost is simply the number of actions taken — depth and cost coincide.

$

### Weighted / variable-cost problems

Cost(s,a,s') can represent distance, travel time, fuel consumed, or monetary price. Here, the number of actions and the total cost can disagree — a route with more segments can still be cheaper overall.

$$g(n) = \Sigma\ Cost(s_i , a_i , s_{i+1})$$

*sum over every step of the path from $s_0$ to n*

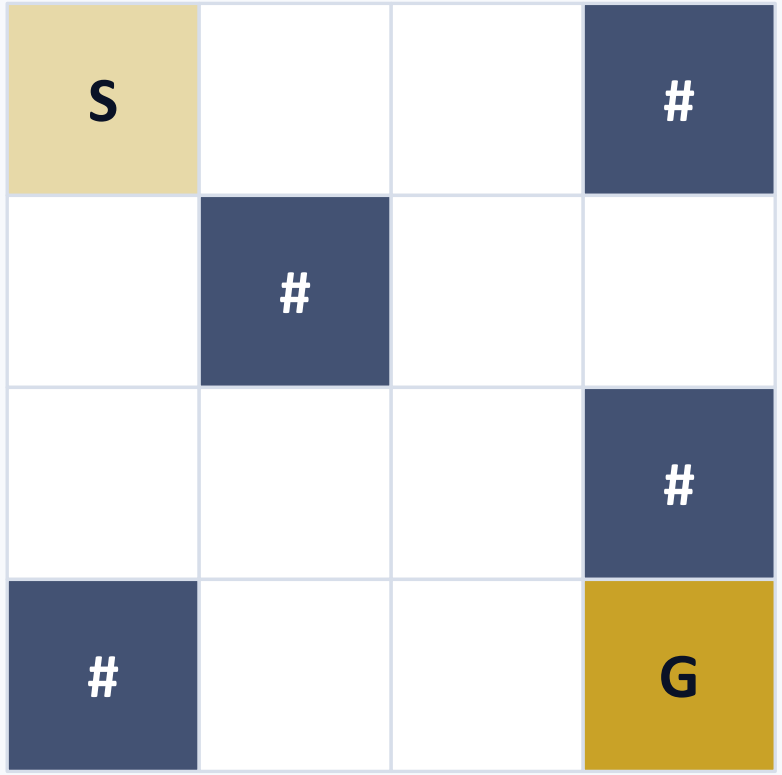
**Why this matters for what's ahead:**

BFS reasons about depth (number of actions).

DFS reasons about depth, exploring one branch at a time.

UCS reasons about g(n) directly — it is the first strategy in this lecture built entirely around path cost, which is why it can find a cheaper solution that BFS would overlook.

# Complete Worked Formulation: A Maze




*S = initial state  G = goal state  # = wall (illegal cell)*

| Component | Formulation for this maze |
|---|---|
| Initial state, $s_o$ | S — the top-left cell |
| Actions, A(s) | UP, DOWN, LEFT, RIGHT (only those staying on the grid and off walls) |
| Transition, Result(s,a) | Move one cell in the chosen direction, if that cell is open |
| Goal test, Goal(s) | current cell = G |
| Step cost, Cost(s,a,s') | 1 for every move (uniform-cost problem) |

# What Is a State?

A state represents the relevant information needed to describe the current situation of the problem. A good state representation contains exactly enough information to determine which actions are legal and what happens next — no more, and no less.

| Problem | What a state captures |
|---|---|
| Maze / grid navigation | (row, column) — the agent's current cell |
| 8-puzzle | The full board configuration — the position of every tile |
| Route finding | The current city — nothing about how the traveler got there |
| Chess | The position of every piece on the board, plus whose turn it is |

**!**

**Sufficiency, not history**

Notice that none of these states record the sequence of actions that produced them. A state need only describe 'where we are now' — the path taken to arrive is tracked separately, by the search node (next section), not by the state itself.

# What Is a State Space?

> ***"The set of all states reachable from the initial state through some sequence of valid actions."***

**A state space is naturally pictured as a graph:**

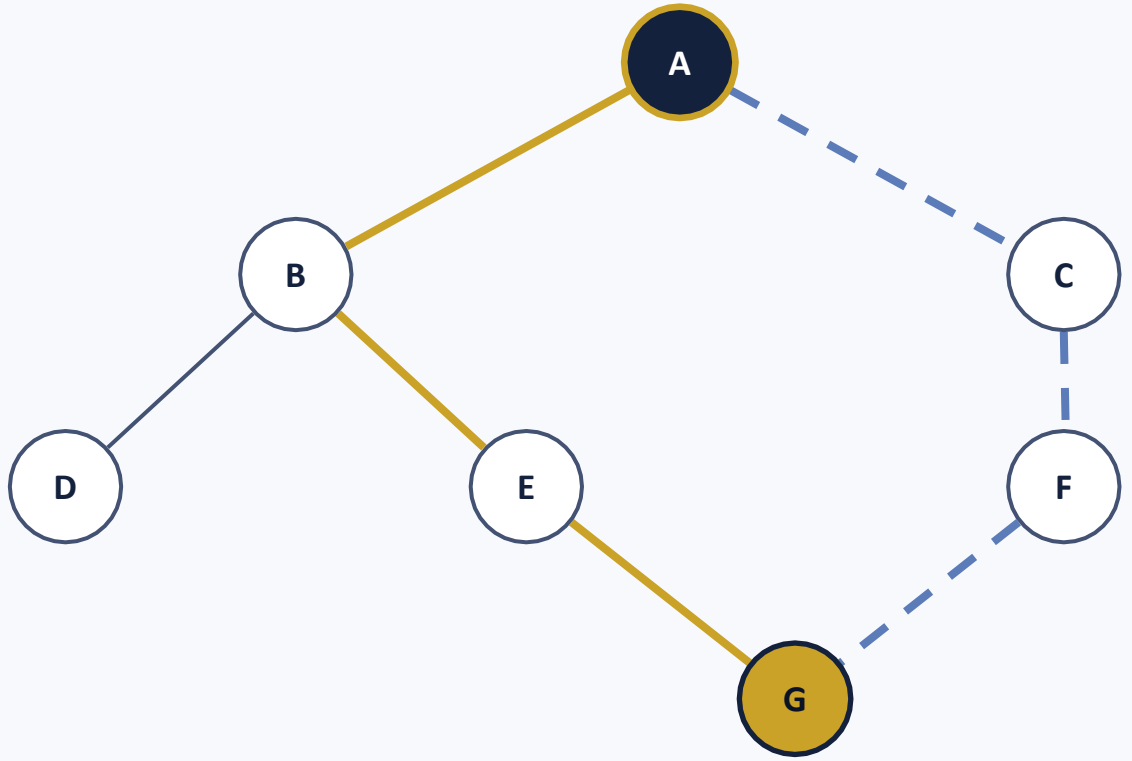
- **Node** = a state the problem could be in
- **Edge** = a transition — an action connecting two states
- **Path** = a sequence of actions from one state to another
- **Goal** = the state (or states) the agent is trying to reach




`Result(s,a) = s'`

*One edge = one action = one call to the transition model.*

# State-Space Graph: A Worked Example




- Path 1: A → B → E → G
- Path 2: A → C → F → G

*A is the initial state; G is the goal. Every edge is a legal action. Both paths reach the goal — which one a search algorithm returns depends entirely on its strategy, as we will see for BFS, DFS, and UCS.*

# State vs. Node vs. Search Tree vs. State-Space Graph

**1** **STATE**

A configuration of the world — 'where we are.' Contains no memory of how we got there.

**2** **SEARCH NODE**

A bookkeeping record the algorithm creates: it wraps a state together with a parent pointer, the action taken, path cost g(n), and depth.

**3** **SEARCH TREE**

The tree of nodes an algorithm actually builds while running — the same state can appear in multiple nodes if reached via different paths.

**4** **STATE-SPACE GRAPH**

The full, problem-defined graph of every state and transition — it exists independently of any particular search algorithm.

*A node contains more information than a bare state:*

`state` `parent` `action` `g(n)` `depth`

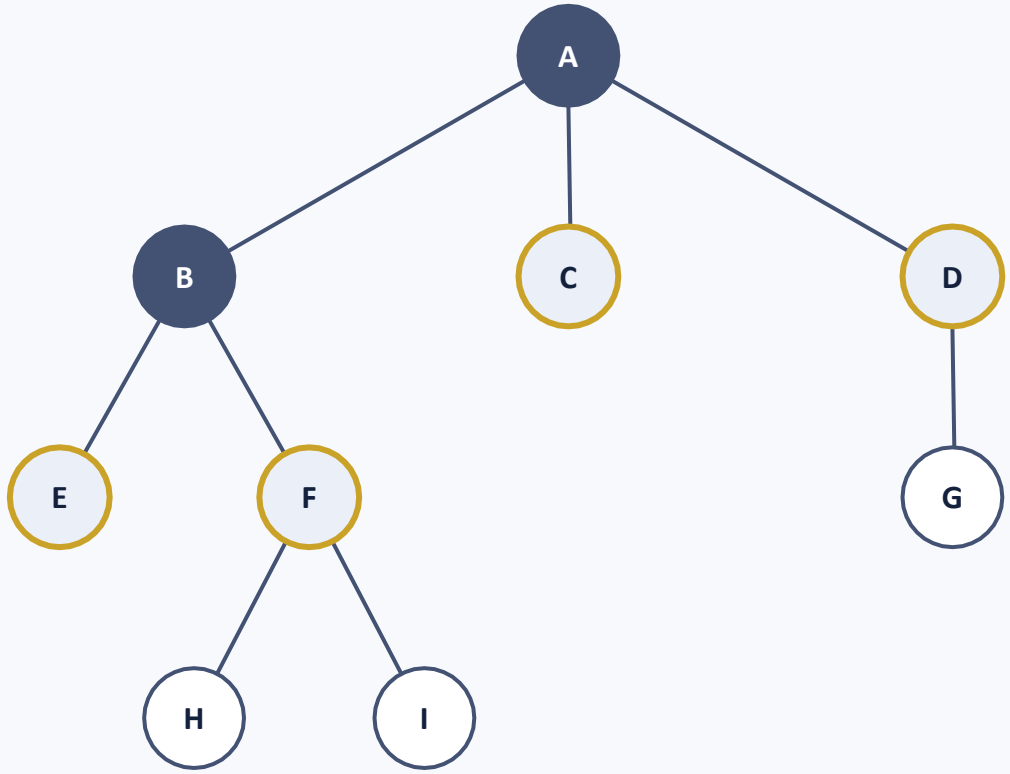
# Frontier and Explored Set

**Frontier (Open List):** nodes that have been generated but not yet expanded.

**Explored (Closed Set):** states that have already been expanded and fully processed.

*Tracking the explored set prevents a graph-search algorithm from expanding the same state twice — without it, cyclic or highly connected state spaces can cause the algorithm to loop or duplicate enormous amounts of work.*

explored frontier unvisited




*Snapshot mid-search: A and B have been expanded (explored); C, D, E, F have been generated but wait in the frontier; G, H, I are not yet discovered.*

# What Is Uninformed Search?

***Uninformed (blind) search uses no estimate of how close a state is to the goal — only the information the problem definition itself provides.***

**An uninformed strategy may use only:**

- The initial state, $s_0$
- The actions available in each state, A(s)
- The transition model, Result(s,a)
- The goal test, Goal(s)
- The path cost, where applicable to the strategy

| | Uninformed search | Informed search |
|---|---|---|
| Extra knowledge used | None beyond the problem definition | A heuristic estimate of distance-to-goal |
| Exploration style | Systematic, by structural position (depth / cost) | Guided toward states that look promising |
| Covered in this lecture? | Yes — BFS, DFS, UCS | Not yet — next lecture (A*, Greedy Best-First) |

# Three Uninformed Search Strategies

## BFS

**Shallowest node first**

`FIFO Queue`

*Expands nodes in the order they were generated — level by level.*

## DFS

**Deepest node first**

`LIFO Stack`

*Commits to one branch and follows it as far as possible before backtracking.*

## UCS

**Lowest path-cost first**

`Priority Queue`

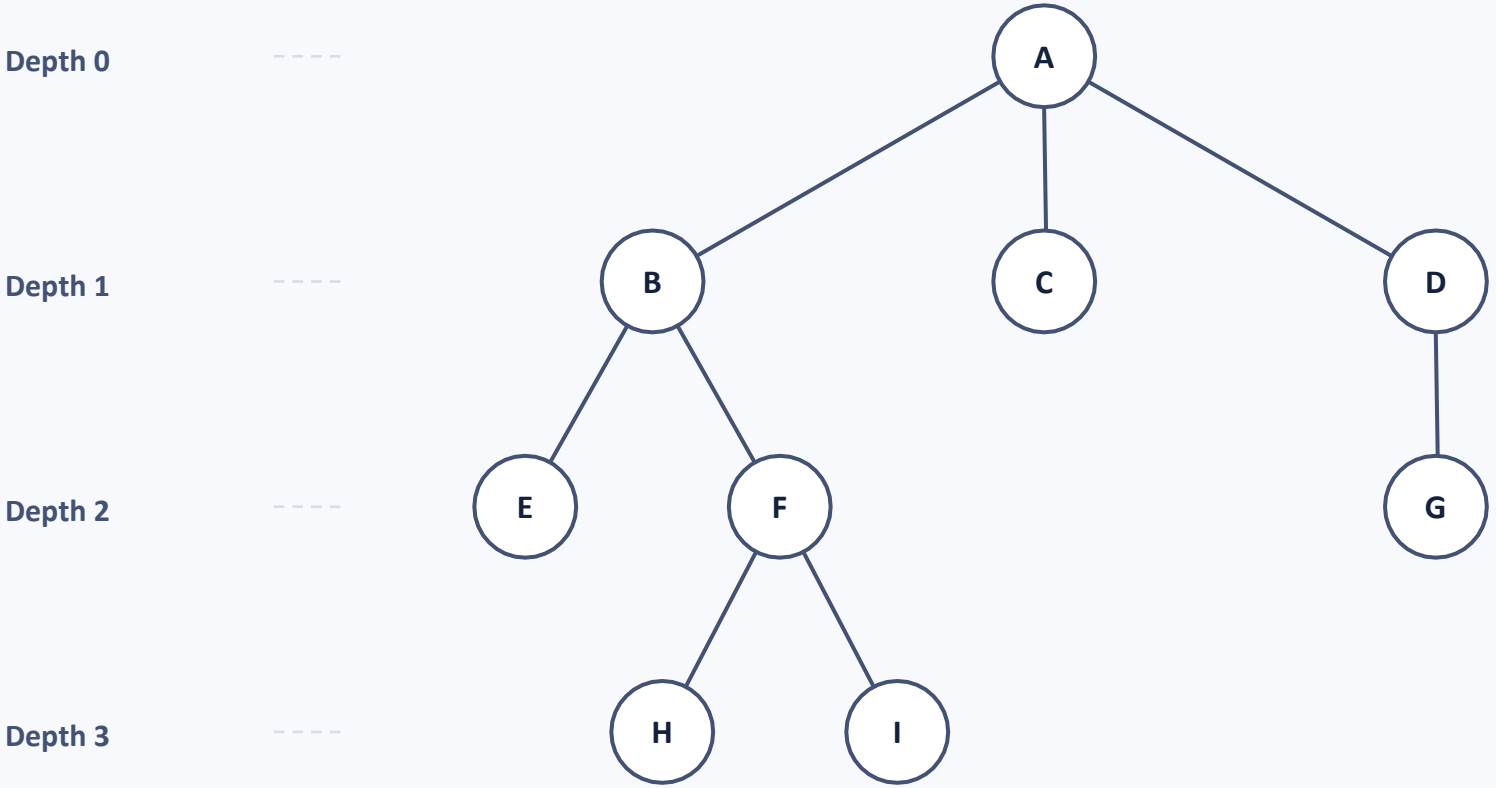
*Always expands the node with the smallest cumulative g(n).*

### Why does the data structure matter so much?

Each strategy is, mechanically, the exact same generic search algorithm — generate successors, add them to the frontier, pick the next node to expand. The ONLY thing that changes between BFS, DFS, and UCS is which data structure holds the frontier, because that data structure determines which node comes out next.

# Breadth-First Search: Concept

**Breadth-First Search expands the shallowest unexpanded node first. Depth is simply the number of actions from the initial state to a node. Because BFS always exhausts every node at the current depth before moving deeper, it explores the state space level by level, like ripples spreading outward from a stone dropped in water.**



# BFS and the FIFO Queue

A FIFO (First-In, First-Out) queue always removes the node that has been waiting the longest — the earliest inserted node leaves first. Because BFS inserts each depth's nodes only after finishing the previous depth, FIFO order automatically enforces breadth-first, level-by-level expansion.

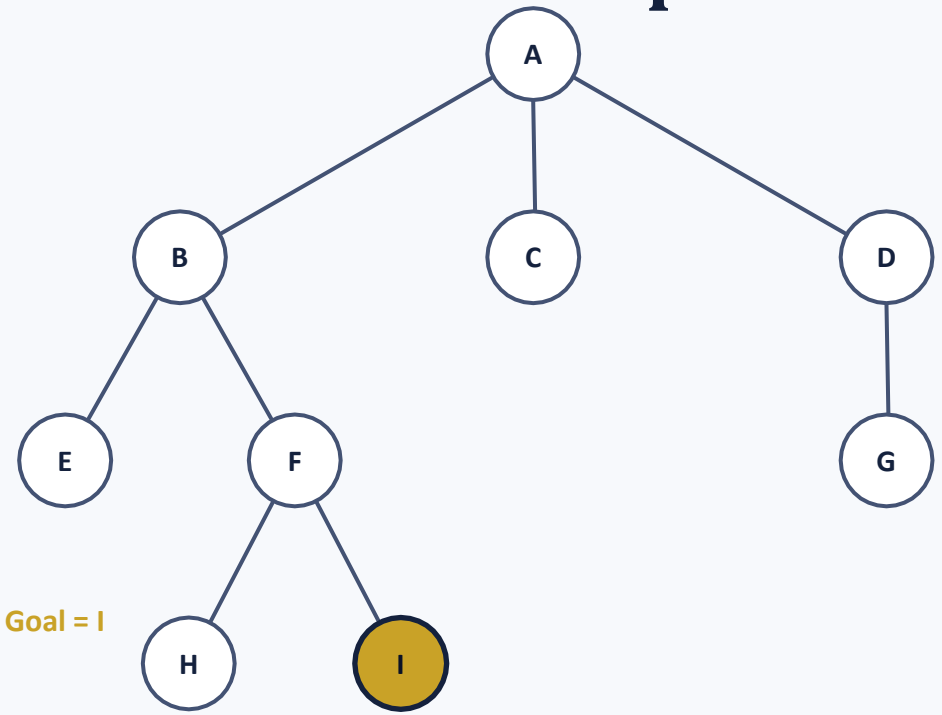
| Frontier state | Contents (front → back) |
|---|---|
| Initial | [ A ] |
| After expanding A | [ B, C, D ] |
| After expanding B | [ C, D, E, F ] |
| After expanding C | [ D, E, F ] |

→

### The earliest-inserted node is removed first

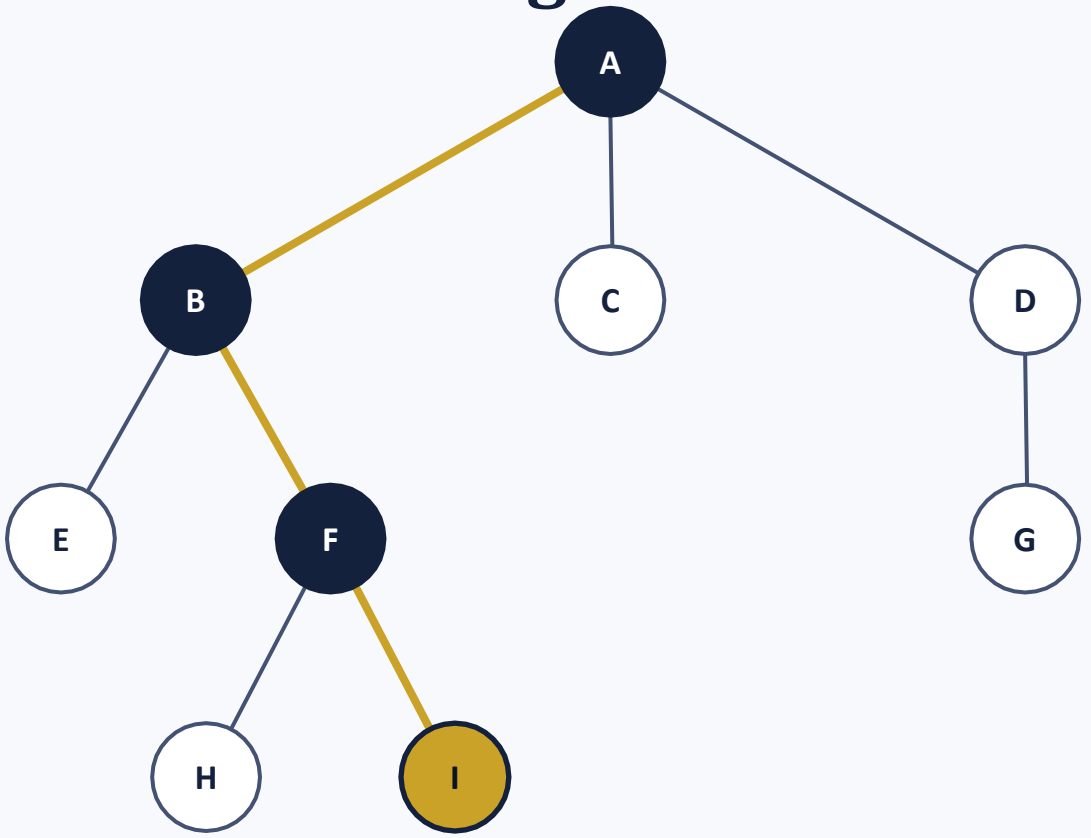
Notice B was inserted before C and D — so once A finishes expanding, B is the very next node removed and expanded, even though C and D are still waiting. This is exactly the FIFO discipline: enqueue at the back, dequeue from the front.

# BFS Worked Example — Full Trace




| Step | Expanded | Frontier (after) | Explored |
|---|---|---|---|
| 1 | A | B, C, D | A |
| 2 | B | C, D, E, F | A, B |
| 3 | C | D, E, F | A, B, C |
| 4 | D | E, F, G | A, B, C, D |
| 5 | E | F, G | A, B, C, D, E |
| 6 | F | G, H, I | A, B, C, D, E, F |
| 7 | G | H, I | A, B, C, D, E, F, G |
| 8 | H | I | A, B, C, D, E, F, G, H |
| 9 | I — GOAL | — | A, B, C, D, E, F, G, H, I |

# BFS Solution and Path Length




**Solution path:**

**A → B → F → I**

Actions taken: 3

States visited: 4

*Path length = number of ACTIONS, not the number of states.*

*Because every step costs 1 in this tree, BFS's shallowest-first strategy guarantees this is a solution with the fewest possible actions — no path from A to I is shorter than 3 actions.*

# BFS: Completeness and Optimality

| | |
|---|---|
| YES | Completeness — under the standard finite-branching-factor assumption |
| YES | Optimality — when every action has the same (uniform) step cost |

Why complete? Breadth-First Search systematically explores the search space level by level. Under the standard assumption of a finite branching factor, if a solution exists at some finite depth d, BFS will eventually finish exploring every shallower level and reach depth d — so it is guaranteed to find a solution.

Why optimal (under uniform cost)? Because BFS never expands a depth-(k+1) node before every depth-k node has been expanded, the very first goal node it encounters must be at the shallowest depth of any goal — and when every action costs the same, shallowest depth means lowest path cost.

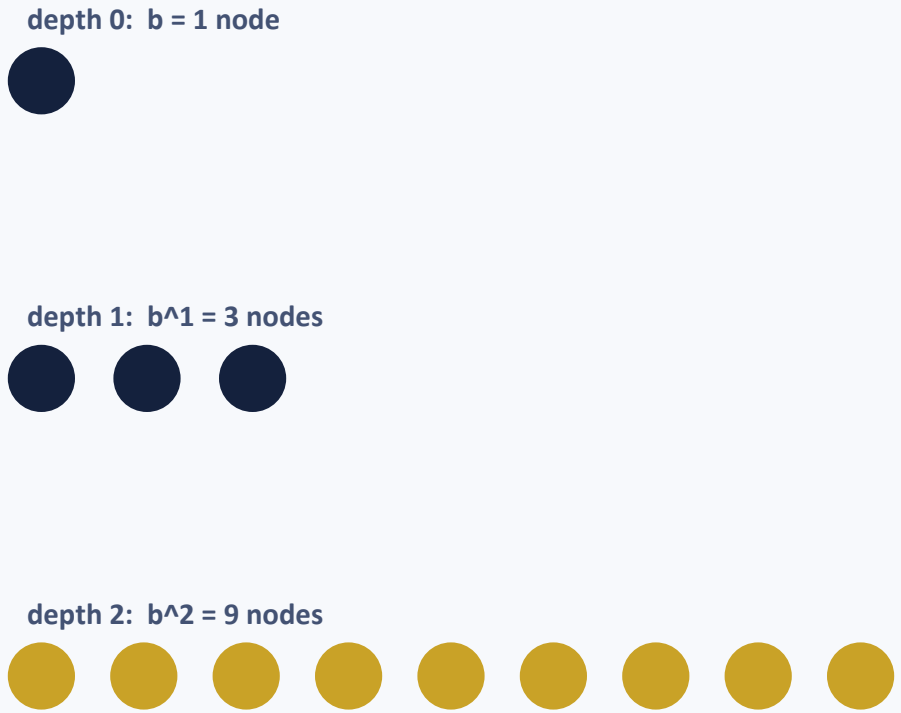
✗ **Misconception: "BFS always finds the cheapest solution."**

False in general — BFS optimizes for FEWEST ACTIONS, not lowest cost. If step costs differ, the shallowest goal may not be the cheapest one (see Slide 30).

# BFS: Time and Space Complexity

$$\text{Time} = O(b^d)$$
$$\text{Space} = O(b^d)$$

- **b** = branching factor — the maximum number of successors of any state
- **d** = depth of the shallowest goal node



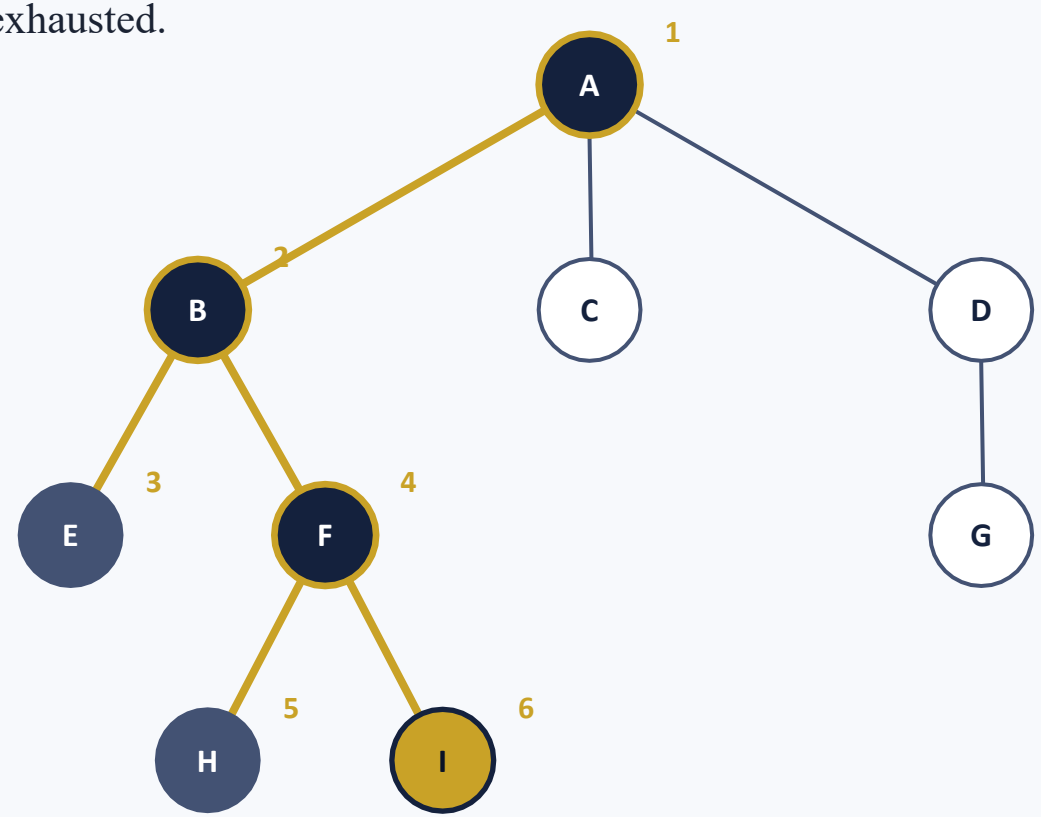

**BFS's defining weakness: memory**

BFS must hold an entire level's worth of nodes in the frontier simultaneously. Because that level grows exponentially with depth, memory — not time — is what makes BFS impractical on large or deep problems.

*With branching factor b = 3, the frontier TRIPLES in size at every depth — this is exponential growth, and it is why BFS memory usage explodes on real-sized problems.*

# Depth-First Search: Concept

Depth-First Search expands the deepest unexpanded node first. Rather than exhausting a level before going deeper, DFS commits immediately to one branch and follows it as far as it possibly can — 'deep before wide' — only backtracking to try a sibling branch once the current one is exhausted.



# DFS and the LIFO Stack

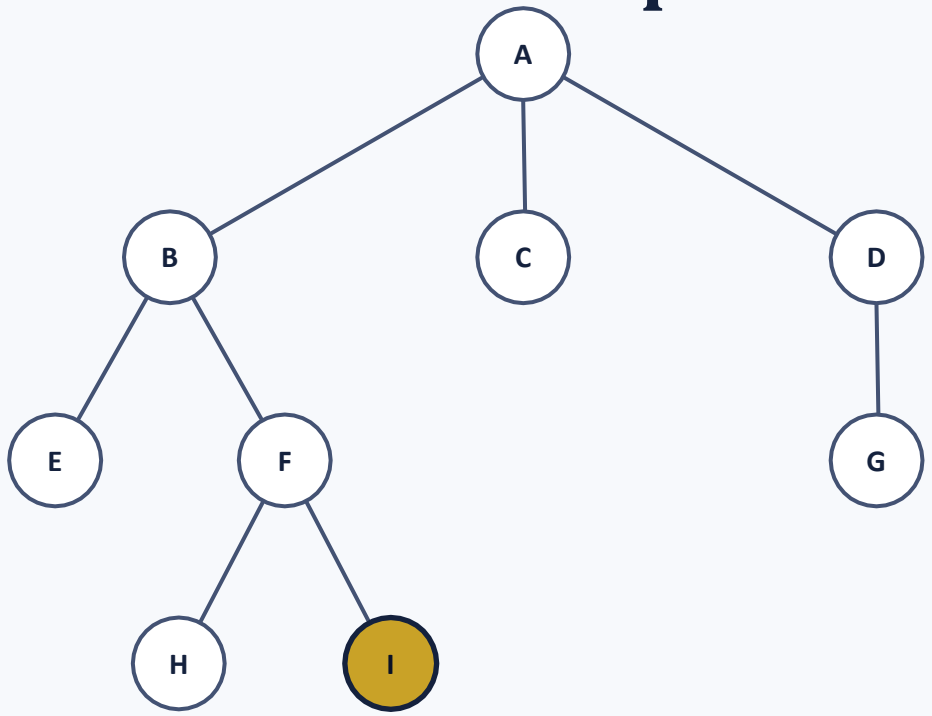
A LIFO (Last-In, First-Out) stack always removes the most recently inserted node first. When a node's children are pushed onto the stack, they are explored immediately — before any previously waiting sibling — which is exactly what produces DFS's depth-first, backtracking behavior.

| Frontier state | Contents (bottom → top) |
|---|---|
| Initial | [ A ] |
| After expanding A | [ D, C, B ] |
| After expanding B | [ D, C, F, E ] |
| After expanding E (leaf — backtrack) | [ D, C, F ] |

↩ **BFS = FIFO · DFS = LIFO**

This single data-structure swap is the entire mechanical difference between the two algorithms — both use the exact same generic expand-and-generate procedure otherwise.

# DFS Worked Example — Full Trace




Goal = I · same tree as the BFS example

| Step | Expanded | Stack (bottom→top, after) | Explored |
|---|---|---|---|
| 1 | A | D, C, B | A |
| 2 | B | D, C, F, E | A, B |
| 3 | E (leaf) | D, C, F | A, B, E |
| 4 | F | D, C, I, H | A, B, E, F |
| 5 | H (leaf) | D, C, I | A, B, E, F, H |
| 6 | I — GOAL | D, C | A, B, E, F, H, I |

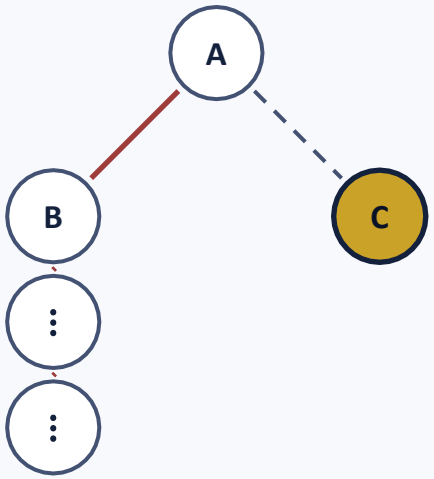
*Successor ordering matters: DFS visited only 6 of the 9 nodes before finding I — but this depends entirely on the left-to-right ordering we chose for pushing children.*

# DFS: Completeness and Optimality

**NO** Completeness — NOT guaranteed on infinite or cyclic state spaces (tree search)

**NO** Optimality — NO. DFS has no mechanism that favors cheaper or shallower solutions

**Why can DFS fail to terminate?**




*An infinite (or very deep) branch traps DFS forever — it never backtracks to try the goal on the other side.*

✗

**Two safeguards, two limits**

Tracking an explored set (graph search) prevents DFS from looping on CYCLES. It does NOT fix an infinite or extremely deep acyclic branch — DFS can still commit to a very long, ultimately fruitless path before a shallow, cheap goal elsewhere is ever tried. This is also why DFS is not optimal: it stops at the first goal it reaches along its current branch, with no comparison to any other path's depth or cost.

# DFS: Time and Space Complexity

$$\text{Time} = O(b^m)$$
$$\text{Space} = O(b \cdot m)$$

- $m$ = maximum depth of the state space (or search tree)

| | BFS | DFS |
|---|---|---|
| Time | $O(b^d)$ | $O(b^m)$ |
| Space | $O(b^d)$ | $O(b \cdot m)$ |
| Growth pattern | Exponential in solution depth d | Linear in maximum depth m |

✓ **DFS's practical appeal: linear memory**

DFS only needs to remember the single path currently being explored, plus the unexplored siblings along that path — not an entire level. This is why DFS is often preferred when memory is tightly constrained, even though it sacrifices completeness and optimality guarantees.

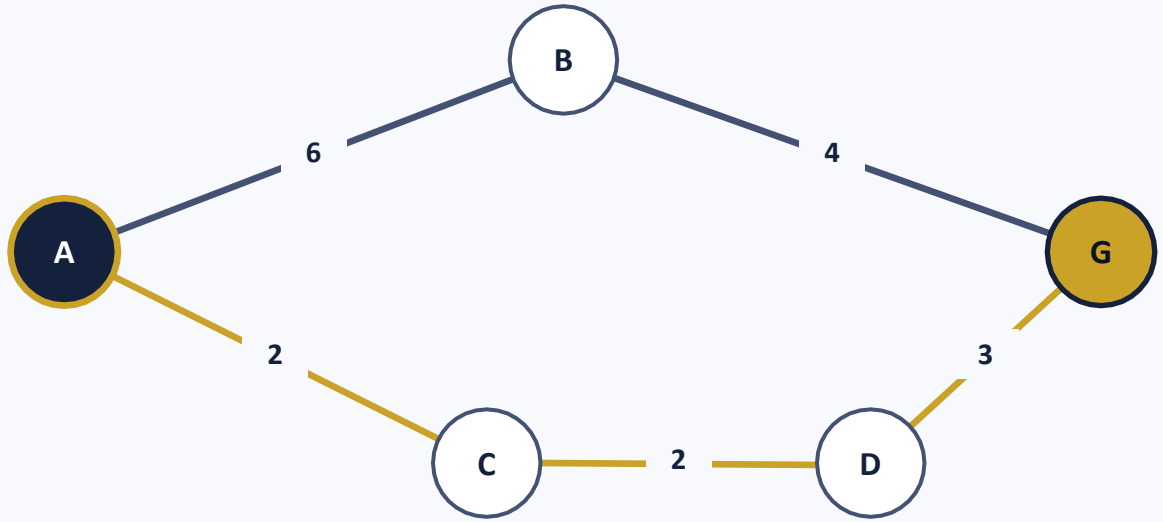
Caution: this is O(b·m) in the standard model — actual memory still depends on implementation details (e.g., how successors are generated and stored), so treat this as a bound, not a promise.

! **Misconception: "DFS is always faster than BFS."**

Not necessarily true. Time complexity depends on m, the MAXIMUM depth, which can be far larger than d, the shallowest solution's depth — especially if the solution is shallow but the tree extends much deeper elsewhere. DFS's advantage is memory, not a general time guarantee.

# Why BFS Is Not Enough

BFS prefers the path with fewer edges. But when actions have different costs, the path with fewer edges is not always the cheaper one — and BFS has no way to notice.




**A → B → G**
2 edges · cost = 10

**A → C → D → G**
3 edges · cost = 7

**BFS would return A → B → G — the more expensive path**

BFS terminates as soon as it reaches G at the shallowest depth (2 edges), never considering that a 3-edge alternative is actually cheaper (cost 7 vs. cost 10). We need a strategy that reasons about cumulative cost directly — this is exactly the motivation for Uniform Cost Search.

# Uniform Cost Search: Concept

***UCS expands the node with the smallest cumulative path cost g(n) — never the shallowest, and never the deepest.***

**BFS chooses:**

**the SHALLOWEST node**

**DFS chooses:**

**the DEEPEST node**

**UCS chooses:**

**the CHEAPEST node — smallest g(n)**

UCS still uses the exact same generic search skeleton as BFS and DFS — generate successors, add to the frontier, pick the next node to expand. Only the selection rule changes: instead of a queue's insertion order or a stack's recency, UCS always removes whichever frontier node currently has the lowest g(n).

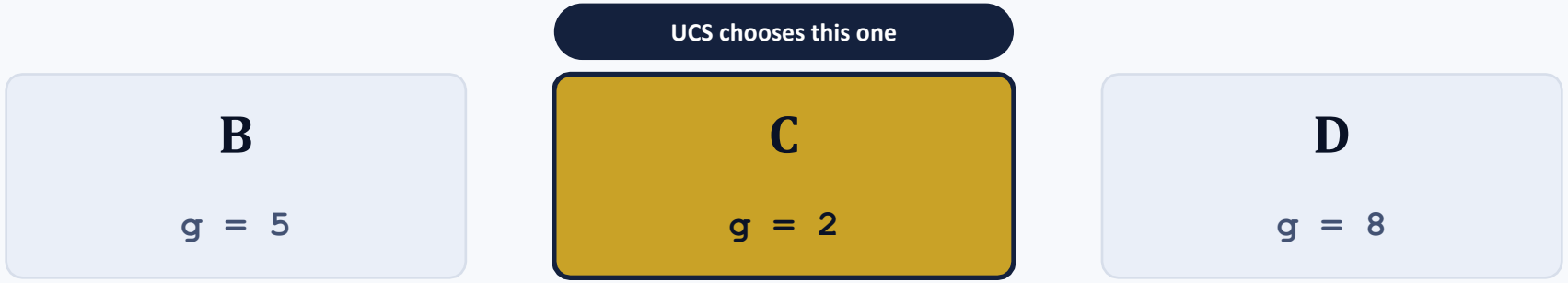
g(n)

**Path cost accumulates from the root**

g(n) is not the cost of the last action alone — it is the running SUM of every action cost from the initial state $s_0$ to node n. UCS compares these running totals across the whole frontier, not just locally.

# UCS and the Priority Queue

A priority queue always removes the element with the smallest priority value — here, the smallest g(n). Unlike FIFO or LIFO, removal order has nothing to do with WHEN a node was inserted; it depends entirely on cumulative cost.

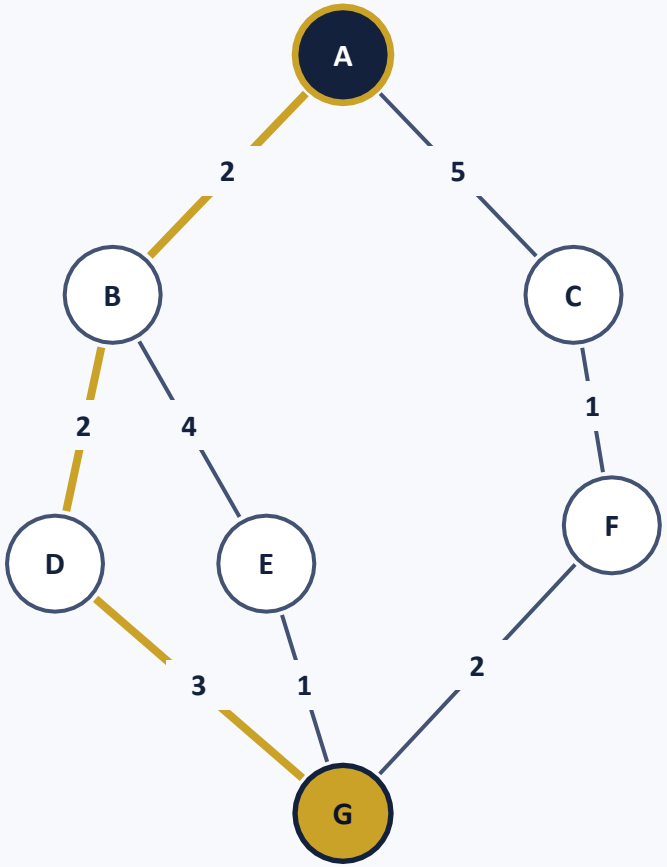



min

**Frontier = { B: g=5, C: g=2, D: g=8 } → expand C**

Even though B and D might have been generated earlier or later, none of that matters to a priority queue — only the g(n) value decides. Because g(n) is a running total from $s_0$, this comparison is always fair, regardless of how many actions each path took to accumulate its cost.

# UCS Worked Example — Weighted Graph




| Step | Expanded | g(n) | Frontier {node:g} | Explored |
|---|---|---|---|---|
| 1 | A | 0 | B:2, C:5 | A |
| 2 | B | 2 | C:5, D:4, E:6 | A, B |
| 3 | D | 4 | C:5, E:6, G:7 | A, B, D |
| 4 | C | 5 | E:6, G:7, F:6 | A, B, D, C |
| 5 | E | 6 | F:6, G:7 | A, B, D, C, E |
| 6 | F | 6 | G:7 | A, B, D, C, E, F |
| 7 | G — GOAL | 7 | — | A, B, D, C, E, F, G |

**Cheapest path found: A → B → D → G total cost = 2 + 2 + 3 = 7**

# UCS: Completeness and Optimality

**YES** Completeness — under standard positive step-cost assumptions

**YES** Optimality — guaranteed under those same standard conditions

Why complete? As long as every step cost is a positive value bounded away from zero, the cumulative cost g(n) strictly increases along every path. UCS therefore cannot loop forever on a bounded-cost search — it will eventually reach and expand any goal at finite cost.

Why optimal? UCS expands nodes in strictly non-decreasing order of g(n). The first time a goal node is popped for expansion, no node remaining in the frontier — and therefore no undiscovered path — can possibly have a lower cost. That first-popped goal must be the cheapest.

**=** **From Slide 33: two equally cheap paths existed**

A-B-D-G and A-B-E-G both cost exactly 7. UCS returns whichever is confirmed first (A-B-D-G here) — optimality guarantees a CHEAPEST path, not a uniquely determined one when ties occur.

**≠** **Cheapest ≠ shallowest**

UCS guarantees the least-cost solution — not the solution with the fewest actions. A deeper (more edges) solution is returned whenever it is cheaper, exactly as in Slide 30.

# Completeness vs. Optimality

### ∃ Completeness

"Will the algorithm find A solution, if one exists at all?" This asks only about SUCCESS — it says nothing about the quality of what is found. An algorithm can be complete yet return a poor solution.

### min Optimality

"Will the algorithm find the BEST (least-cost) solution?" This is a strictly stronger requirement — it asks about quality, and it can only be asked of an algorithm that is already complete.

| | Complete? | Optimal? | Illustration |
|---|---|---|---|
| BFS | Yes | Yes, if uniform cost | Finds shortest-depth solution first |
| DFS | Not guaranteed | No | May return the first solution reached, regardless of cost |
| UCS | Yes | Yes | Finds least-cost solution, whatever its depth |

*These are two independent properties — an algorithm can have either, both, or neither. Confusing 'finds a solution' with 'finds the best solution' is one of the most common errors in evaluating search strategies.*

COMPARISON AND ANALYSIS

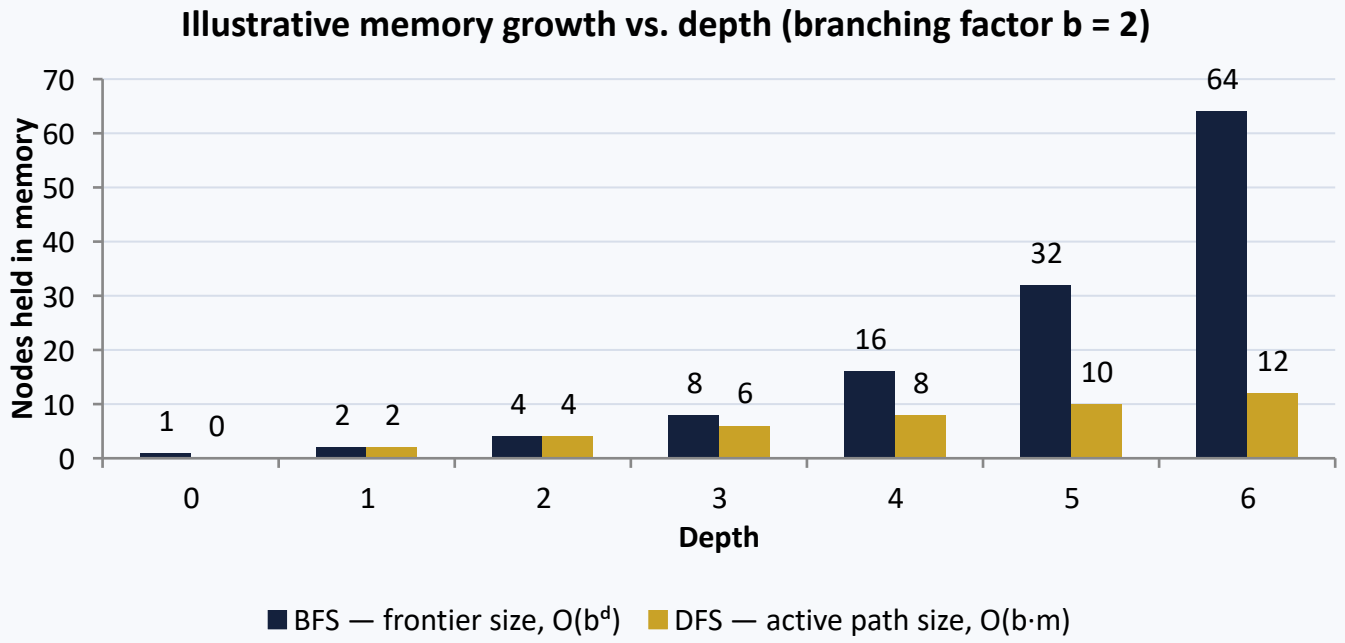
# BFS vs. DFS vs. UCS — Master Comparison

*Note: UCS complexity is best expressed relative to C* (optimal solution cost) and ε (smallest step cost): O(b^(1+⌊C*/ε⌋)) — never simply $O(b^d)$, since UCS does not reason about depth at all.*

| Property | BFS | DFS | UCS |
|---|---|---|---|
| Expansion rule | Shallowest node first | Deepest node first | Lowest g(n) first |
| Data structure | FIFO queue | LIFO stack | Priority queue (by g(n)) |
| Complete? | Yes (finite b) | Not guaranteed (infinite/cyclic spaces) | Yes (positive step costs) |
| Optimal? | Yes, if step cost is uniform | No | Yes, under positive step costs |
| Time complexity | $O(b^d)$ | $O(b^m)$ | Cost-dependent — not simply $O(b^d)$ |
| Space complexity | $O(b^d)$ | $O(b \cdot m)$ | Cost-dependent, can approach $O(b^d)$-scale |
| Main strength | Guarantees shortest #actions | Very low memory relative to depth | Guarantees lowest total cost |
| Main weakness | Exponential memory | No optimality or completeness guarantee | Can expand many nodes if costs are close/small |
| Typical use case | Unweighted shortest-path problems | Memory-limited exploration, deep search trees | Weighted problems (routing, cost-sensitive planning) |

# Memory Considerations

Search algorithms are frequently memory-bound before they are time-bound: a computation that would eventually finish can still fail in practice because the frontier or explored set exceeds available memory. Two quantities matter most: maximum frontier size and explored-set size.




*Values are computed directly from the $O(b^d)$ and $O(b \cdot m)$ formulas with b = 2 — an illustration of the formulas, not measured experimental data.*

**BFS**

Must hold an entire level in the frontier at once. Grows exponentially with depth — the dominant practical limitation of BFS.

**DFS**

Holds only the current path plus unexplored siblings. Grows linearly with depth — DFS's key practical advantage.

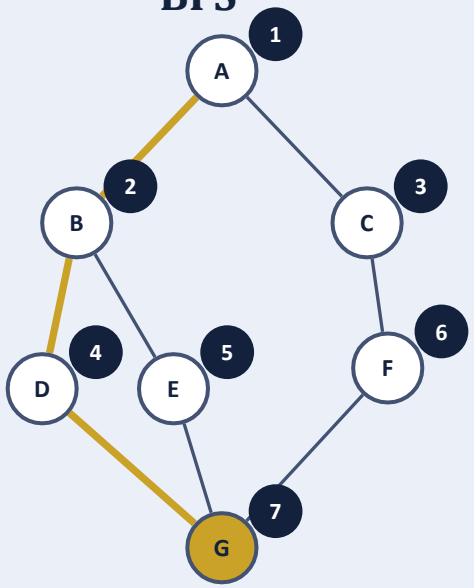
**UCS**

Can hold many competing partial paths whenever costs are close together — frontier size depends on the cost structure of the problem, not depth alone.

COMPARISON AND ANALYSIS

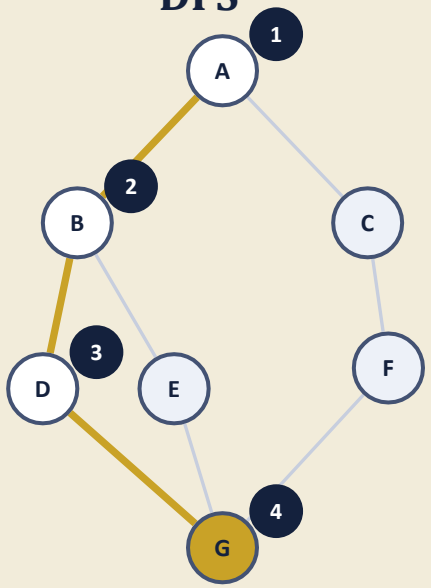
# Same Problem, Three Algorithms

## BFS



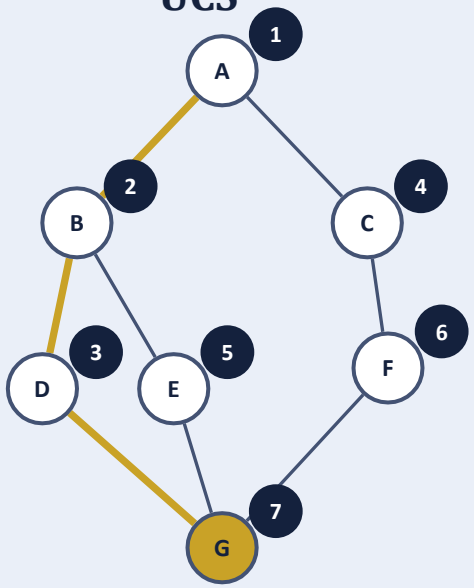

7 expansions · path A-B-D-G · cost 7

## DFS




4 expansions · path A-B-D-G · cost 7

## UCS



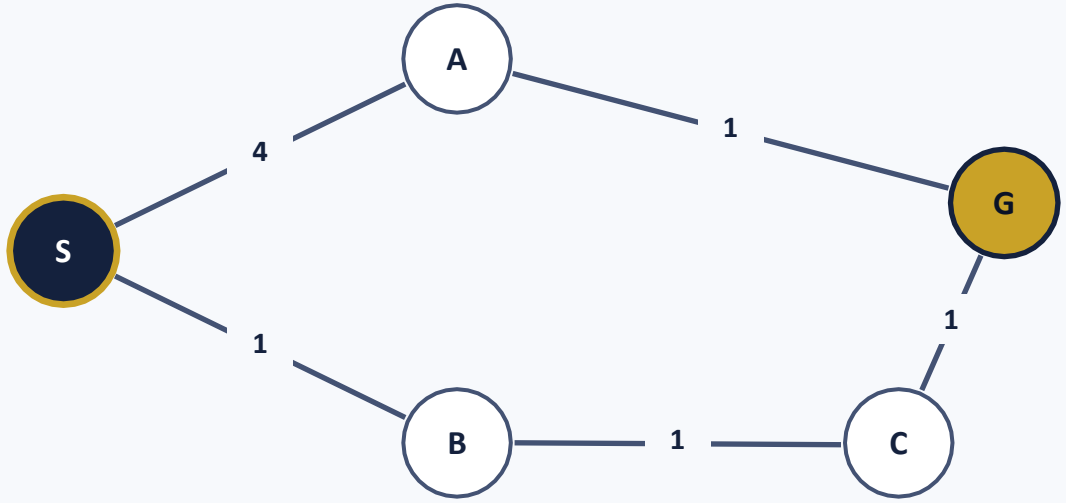

7 expansions · path A-B-D-G · cost 7

**All three agree here — but for very different reasons, and that won't always happen**

In this graph, every root-to-goal path happens to use 3 edges, so BFS's fewest-edges criterion accidentally matches UCS's lowest-cost criterion. DFS reached the same answer only because of left-to-right ordering luck (see Slide 28) — none of this is a general guarantee.

CLASSROOM CHALLENGE

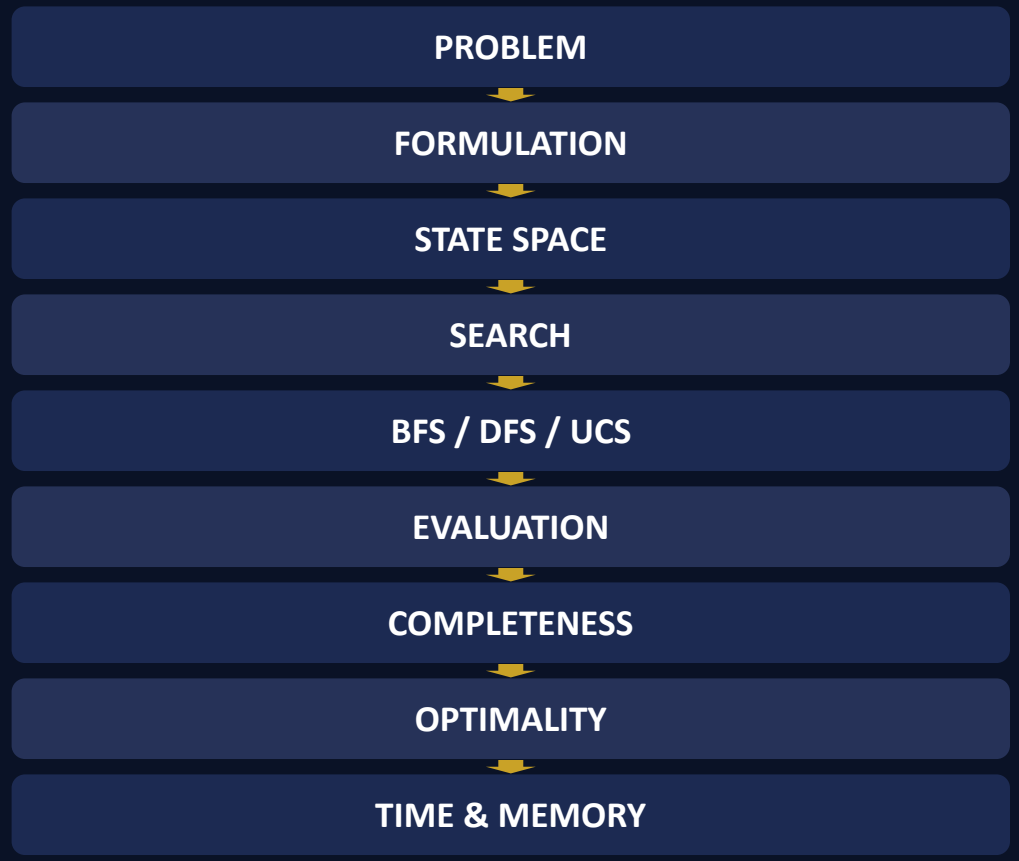
# Trace It Yourself — 3 to 5 Minutes




*All edges are directed away from S. Weights are shown on each edge.*

- What is the initial state? What is the goal?
- What actions are legal in state S?
- What is BFS's order of expansion?
- What is DFS's order of expansion (leftmost branch first)?
- What is UCS's order of expansion?
- Which path has the fewest edges (is 'shortest')?
- Which path has the lowest total cost (is 'cheapest')?
- Are the shortest and cheapest paths the same path here?
- Which algorithm(s) are guaranteed to find the cheapest path?
- Which algorithm is likely to need the most memory here, and why?

## From Formulation to Search — What We Covered




**Every algorithm we studied today shares one skeleton:**

generate successors, add them to a frontier, and repeatedly remove and expand one node — differing only in which node the frontier hands back.

**BFS** picks the shallowest node (FIFO).
**DFS** picks the deepest node (LIFO).
**UCS** picks the cheapest node (priority queue) — and is the only one of the three that is optimal without a uniform-cost assumption.

> ***"If the search space becomes extremely large, how can AI use additional knowledge about the problem to search more intelligently?"***

**NEXT LECTURE → From Uninformed Search to Informed Search**

**Question:**

Write a simple reflex agent for a **vacuum-cleaner environment** using **IF–THEN rules**.

The agent can sense:

- Its current location: **A or B**
- Whether the current location is **Dirty or Clean**

Write **four rules** that specify what the agent should do.

**Example format:**

```
IF location = A AND status = Dirty
THEN action = Suck
```